

# Choosing an Activation

---

Top layer - Output layer

## 1. Linear Units for Gaussian Output Distributions

---

Linear output units (no activation) predict the **mean of a Gaussian (normal) distribution**, making them ideal for **regression tasks**. The model assumes the target variable follows a Gaussian distribution with a fixed or learned variance.

### Gaussian Distribution Basics

- **Probability Density Function (PDF):**

$$p(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

- $\mu$ : Mean (predicted by the model)
- $\sigma^2$ : Variance (fixed or learned)

- **Negative Log-Likelihood (NLL):**

$$-\log p(y) = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y - \mu)^2}{2\sigma^2}$$

- If  $\sigma^2$  is fixed, minimizing NLL = minimizing **Mean Squared Error (MSE)**

### Author's Message

*"For regression, use linear outputs to predict the mean of a Gaussian distribution. MSE is the natural loss function here, as it's equivalent to the negative log-likelihood of the Gaussian. This ensures principled, gradient-friendly learning."*

### Example

- **Task:** Predict house prices
  - The model outputs  $\mu$  (mean price) for given input features
  - Interpretation: Learns to predict the **average house price** for each input configuration
- **Advanced Use Case:** Heteroscedastic Regression
  - Predicts **both mean and variance** to model uncertainty
  - Uses **log-variance** for numerical stability

## Key Insight

- Linear outputs + MSE = **maximum likelihood estimation (MLE)** for Gaussian distributions
- Avoids gradient saturation (unlike sigmoid/softmax)

## 2. Sigmoid Units for Bernoulli Output Distributions

---

Sigmoid units predict the **probability of a binary outcome** (Bernoulli distribution), making them standard for **binary classification**. The sigmoid function ensures outputs are valid probabilities (0 to 1).

### Bernoulli Distribution Basics

- **Probability Mass Function (PMF):**

$$P(y) = p^y(1 - p)^{1-y}$$

- $y \in \{0, 1\}$  (binary outcome)
- $p$ : Probability of  $y = 1$

- **Negative Log-Likelihood (NLL):**

$$-\log P(y) = -[y \log(p) + (1 - y) \log(1 - p)]$$

- This is the **binary cross-entropy loss**

## Sigmoid Function

- **Formula:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Properties:**
  - Maps any real  $z$  to  $(0, 1)$
  - Avoids gradient saturation when paired with cross-entropy

## Author's Message

*"For binary classification, use sigmoid outputs to predict Bernoulli probabilities. Binary cross-entropy is the natural loss function, as it matches the negative log-likelihood of the Bernoulli distribution. Avoid MSE with sigmoid—it causes vanishing gradients!"*

## Example

- **Task:** Email spam detection
  - Model outputs  $P(\text{spam} = 1 \mid \text{email features})$
  - Interpretation: Probability score (e.g., 0.8 = 80% spam)
- **Why Not MSE?**
  - MSE gradients vanish when  $\sigma(z)$  saturates (near 0 or 1)
  - Cross-entropy "undoes" the sigmoid's saturation via the log term

## Key Insight

- Sigmoid + cross-entropy = **MLE for Bernoulli distributions**
- Ensures stable gradients even when predictions are confident (near 0 or 1)

---

## 3. Softmax Units for Categorical Output Distributions

---

Softmax units generalize sigmoid to **multi-class classification**, predicting a **vector of probabilities** that sum to 1 (Categorical distribution). Each output corresponds to the probability of a class.

## Categorical Distribution Basics

- **Probability Mass Function (PMF):**

$$P(y = i) = p_i, \quad \sum_{i=1}^k p_i = 1$$

- $k$ : Number of classes
- $p_i$ : Probability of class  $i$

- **Negative Log-Likelihood (NLL):**

$$-\log P(y) = -\sum_{i=1}^k y_i \log(p_i)$$

- This is the **categorical cross-entropy loss**

## Softmax Function

- **Formula:**

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)}$$

- **Properties:**
  - Outputs are probabilities that sum to 1
  - Avoids gradient saturation for the correct class

## Author's Message

*"For multi-class classification, use softmax outputs to predict Categorical probabilities. Categorical cross-entropy is the natural loss function, as it's the negative log-likelihood of the Categorical distribution. Avoid MSE with softmax—it causes gradient issues!"*

## Example

- **Task:** Handwritten digit classification (0-9)
  - Model outputs  $P(y = i \mid \text{image})$  for each digit  $i$
  - Interpretation: Probability vector (e.g., [0.1, 0.8, 0.1] for digits 0, 1, 2)
- **Numerical Stability**
  - Use  $\text{softmax}(z - \max(z))$  to avoid overflow from large  $z$  values

## Key Insight

- Softmax + cross-entropy = **MLE for Categorical distributions**
  - The log in cross-entropy “undoes” the exp in softmax, preventing gradient saturation
- 

## 4. Other Output Types (Custom Distributions)

---

Neural networks can predict **parameters of any distribution**, not just Gaussian/Bernoulli/Categorical. Examples include:

- Mixture of Gaussians: For multi-modal data
- Heteroscedastic Regression: Predicting mean + variance for uncertainty estimation
- Custom Distributions: Any distribution where you can compute the negative log-likelihood

## Prerequisites

### Mixture of Gaussians

- **Formula:**

$$p(y) = \sum_{i=1}^n w_i \cdot \mathcal{N}(y; \mu_i, \sigma_i^2)$$

- $w_i$ : Weight of the  $i$ -th Gaussian (sums to 1)
- $\mu_i, \sigma_i^2$ : Mean and variance of the  $i$ -th Gaussian
- **Use Case:** Multi-modal regression (e.g., predicting two possible house prices for a neighborhood)

## Maximum Likelihood Estimation (MLE)

- For **any distribution**, the loss function is the **negative log-likelihood** of the data under the model
- Example: For a Gaussian mixture, the loss is:

$$L = -\log \left( \sum_{i=1}^n w_i \cdot \mathcal{N}(y; \mu_i, \sigma_i^2) \right)$$

## Author's Message

*"You're not limited to standard outputs. For advanced tasks (e.g., multi-modal regression), design custom outputs and use the negative log-likelihood of your chosen distribution as the loss function. MLE provides a universal framework for this."*

## Example

- **Task:** Predict house prices in a city with rich and poor neighborhoods
  - **Model Outputs:** Weights ( $w_i$ ), means ( $\mu_i$ ), and log-variances ( $\log \sigma_i^2$ ) for a mixture of Gaussians
  - **Interpretation:** Captures **multiple price modes** (e.g., \$300K and \$1M) for the same input features

## Key Insight

- **Flexibility:** Can model **any distribution** by predicting its parameters and using its NLL as the loss

- **Stability:** Use log-variance (not variance) and softplus (not exp) for numerical stability